B2C | MLE | CONTINU | SEPT 25

Isabelle WUILBEAUX – Sophie BONNEAU – Victor DOSDA

Rapport de projet

Classification et extraction d'information sur un document

Table des matières

1. Contexte et objectif du projet 3
    1.1 Contexte 3
        1.1.1 Cadrage métier 3
        1.1.2 Objectif métier 3
        1.1.3 Gains attendus 4
    1.2 L'objectif du projet 4
    1.3 Notre problématique 4
    1.4 Notre démarche 5
    1.5 Notre Streamlit 5
2. Exploration 6
    2.1 Dataset 1 6
    2.2 Dataset 2 8
3. Modélisation sur Dataset 1 10
    3.1 Méthode sur les features 10
    3.2 Méthode sur le texte et les pixels 11
    3.3 Constat 13
4. Modélisation sur Dataset 2 13
    4.1 Méthode sur les features 13
    4.2 Méthode sur le texte et les pixels 16
        4.2.1 Méthode sur les pixels 16
        4.2.2. Méthode sur le texte 16
    4.3 Méthode sur le CNN – deep learning 21
        4.3.1 Feature Extraction : 3 réseaux pré-entraînés et gelés 21
        4.3.2 Fine-Tuning total: un CNN EfficientNet-B0 22
    4.4 Méthode LLM 24
    4.5 Améliorer le modèle SVM sur TF-IDF caractères 24
        4.5.1 Arbitrer seulement les cas les plus durs (top1 vs top2) 24
        4.5.2 Combiner le texte et l'image : Voting et Stacking 26
        4.5.3 Stacking à 3 modèles : texte + image + CNN 27
    4.6 Constat 29
5. Explicabilité du modèle CNN 29
    5.1 Grad-CAM 29
    5.2 SHAP 33
    5.3 Constat 34
6. Conclusion 34

1. Contexte et objectif du projet

1.1 Contexte

1.1.1 Cadrage métier

Chaque jour, les assureurs reçoivent des documents provenant de sources multiples :

- scans hétérogènes,
- photos prises sur le terrain,
- PDF générés automatiquement,
- pièces jointes issues de systèmes variés.

Cette diversité documentaire représente un défi majeur :

- qualité variable,
- formats incohérents,
- structures différentes,
- informations dispersées,
- risques d'erreurs, de litiges et de non conformité.

Les équipes passent trop de temps à :

- ➢ rechercher des informations dans des documents
- ➢ vérifier des données extraites
- ➢ corriger des erreurs OCR
- ➢ gérer des exceptions
- ➢ re-traiter des documents de mauvaise qualité

Ce fonctionnement crée :

- des coûts opérationnels élevés
- des risques d'erreurs
- une faible capacité de montée en charge
- une expérience utilisateur dégradée

Pour faire face à cette situation, le traitement numérique des documents est à renforcer.

La digitalisation moderne ne consiste plus seulement à dématérialiser, mais à valoriser les données contenues dans ces documents.

1.1.2 Objectif métier

Dans un environnement où les documents arrivent sous des formes multiples — scans, photos, PDF hétérogènes — la digitalisation moderne ne consiste plus seulement à dématérialiser, mais à valoriser les données contenues dans ces documents.

Chaque document devient un actif numérique capable d'alimenter les processus métier, d'améliorer la qualité de décision et de renforcer la performance opérationnelle.

La transformation digitale du traitement documentaire repose sur :

- ★ l'automatisation,
- ★ l'intelligence artificielle multimodale,
- ★ la structuration des données,
- ★ l'intégration fluide dans le système d'information.

1.1.3 Gains attendus

Les entreprises attendent de l'IA des gains à plusieurs niveaux :

➔ Gains métier
- Détection automatique des incohérences et anomalies
- Réduction des litiges et contestations
- Amélioration de la conformité documentaire
- Fiabilisation des décisions métier

➔ Gains opérationnels
- Automatisation de la classification des documents
- Réduction des tâches répétitives et chronophages
- Diminution des erreurs humaines
- Accélération des délais de traitement
- Capacité à absorber des volumes plus importants sans augmenter les effectifs

➔ Gains financiers
- Réduction des coûts de non-qualité
- Diminution des coûts liés aux litiges et contre-expertises
- Optimisation des contrôles prestataires
- Valorisation des données extraites

➔ Gains SI / Processus
- Réinjection automatique des données dans le système d'information
- Intégration fluide dans les workflows existants
- Traçabilité complète du traitement documentaire
- Industrialisation du processus de bout en bout # Gains attendus pour l'entreprise

1.2 L'objectif du projet

L'objectif du projet est de classer automatiquement des documents administratifs scannés à partir de leur image, sans métadonnée et sans nom de fichier informatif.

Cet objectif entre dans la volonté de transformer les documents en un réservoir structuré d'information exploitable, grâce à un processus d'automatisation intelligente capable de :
- comprendre le document
- analyser sa structure
- classifier le document
- extraire le maximum d'information utile

1.3 Notre problématique

Il s'agit d'un problème de classification multi-classe. Un document porte deux types d'information :

- son texte : le vocabulaire (« invoice », « total », « subject »…) ;
- sa mise en page : tableaux, colonnes, en-têtes, zones vides.

Laquelle de ces deux sources permet le mieux de reconnaître un document et gagne-t-on à les combiner ?

1.4 Notre démarche

Nous avons avancé par étapes.

Nous avons d'abord catalogué et comparé des modèles de Machine Learning sur un dataset simple puis sur un dataset plus réaliste.

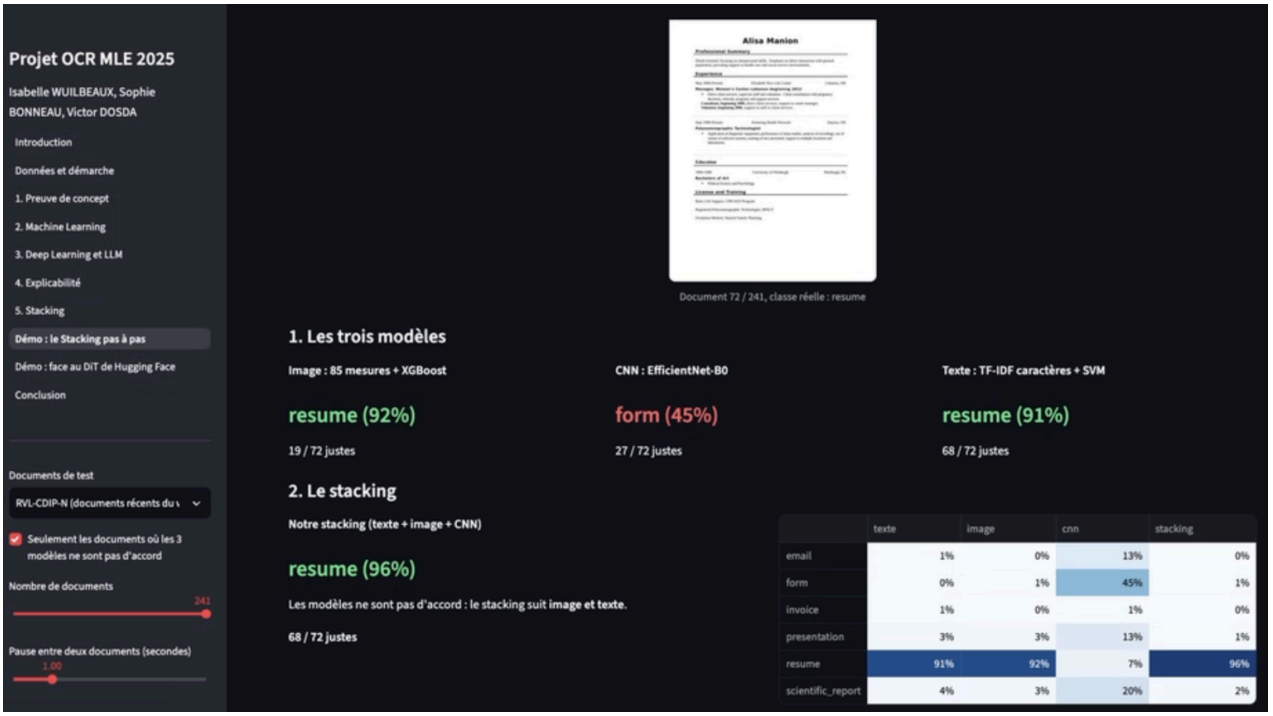
Nous avons ensuite utilisé des modèles de Deep Learning.

Nos classes étant équilibrées, nous utilisons l'accuracy comme métrique principale. La matrice de confusion la complète pour voir quelles classes sont confondues entre elles.

Toutes nos lignes de code sont disponibles sur https://github.com/DataScientest-Studio/sep25_mle_ocr

1.5 Notre Streamlit

Tout le code du Streamlit est disponible sur https://github.com/DataScientest-Studio/sep25_mle_ocr



2. Exploration

Nous avons travaillé sur 2 jeux de données, progressivement plus grands et plus proches d'un cas réel.

	Dataset 1	Dataset 2
Source	Kaggle	Hugging Face (RVL-CDIP)
Nombre de classes	3	6
Classes	email, resume, scientific_publication	email, form, invoice, presentation, resume, scientific_report
Taille	165 images (55 par classe)	30 000 images (5 000 par classe), puis 60 000 (10 000 par classe)
Dossier	document-classification-dataset_1	rvl_cdip_filtered_dataset_2 (30 000), rvl_cdip_filtered_dataset_3 (60 000)

Dans les deux cas, les images sont rangées dans un dossier par classe.
Le nom du dossier sert d'étiquette (label) :

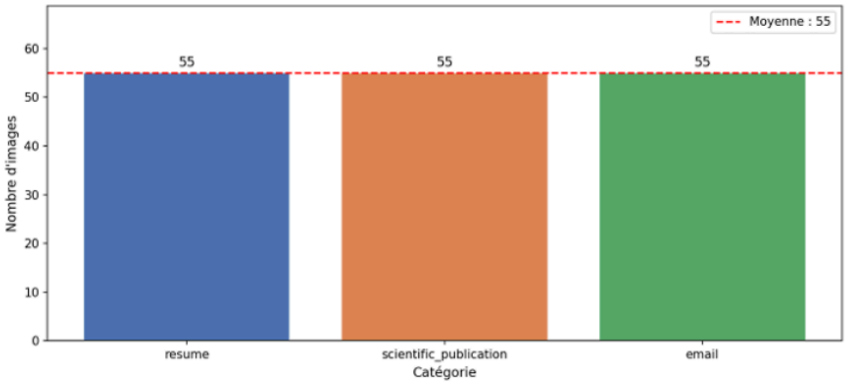
```
root_dir/
  ├── class1/
  │    ├── image1.png
  │    ├── image2.png
  │    └── ...
  └── class2/
       ├── image1.png
       ├── image2.png
       └── ...
  `...
```

2.1 Dataset 1

Nous avons démarré le projet avec le jeu de données proposé : 165 documents et 3 classes bien distinctes.

Ci-dessous le graphique représentant la distribution du dataset 1 :

Fig. 1 — Distribution des catégories de documents



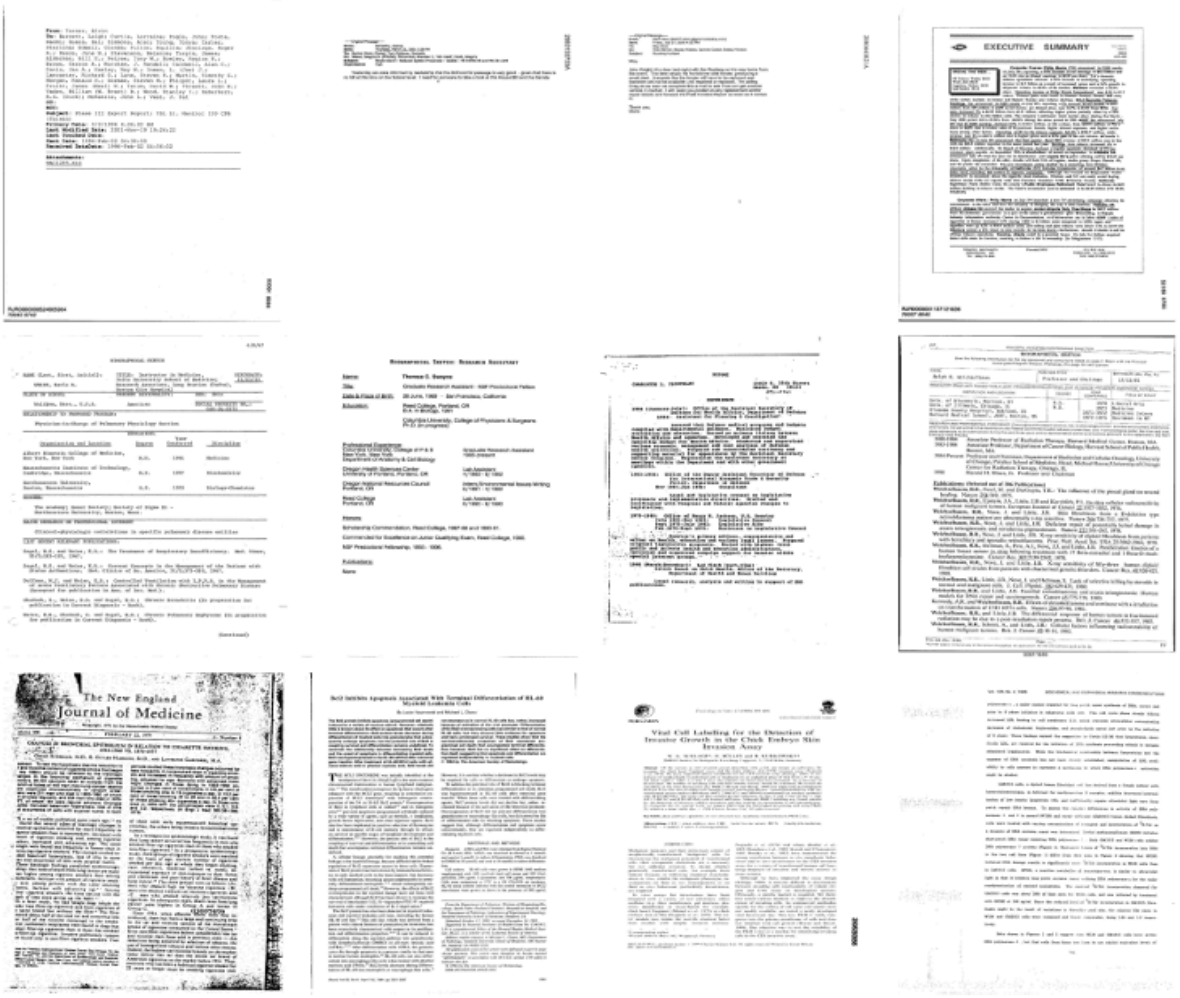
Le nombre de documents par classe est parfaitement équilibré : 55 resume / 55 scientific_publication / 55 email.

Avoir un dataset équilibré évitera qu'un modèle triche en prédisant toujours la classe majoritaire. L'accuracy mesurée sera directement comparable entre classes.
Il n'y a pas de nettoyage à faire sur ce dataset.

Ce dataset est petit. Avec 20 % des images gardées pour le test, il reste seulement 33 documents de test : une seule erreur fait varier l'accuracy d'environ 3 points.

L'affichage de 4 documents par classe montre des mises en page très différentes :
email : un en-tête (From, To, Subject) suivi d'un texte court ;
resume : des rubriques structurées, souvent en tableau ;
scientific_publication : des colonnes de texte dense.

Cette observation pourrait nous amener à penser que même des méthodes simples arriveront à bien séparer ce dataset.



Source : 2_exploration/2.1_dataset_1.ipynb

Dans notre repo vous trouverez une partie ***Annexe (exploration Dataset 1)*** qui décrit toutes les tâches d'exploration que nous avons faites sur le Dataset 1.

2.2 Dataset 2

Pour nous rapprocher d'un cas réel, nous avons choisi un second dataset, plus grand et plus difficile.

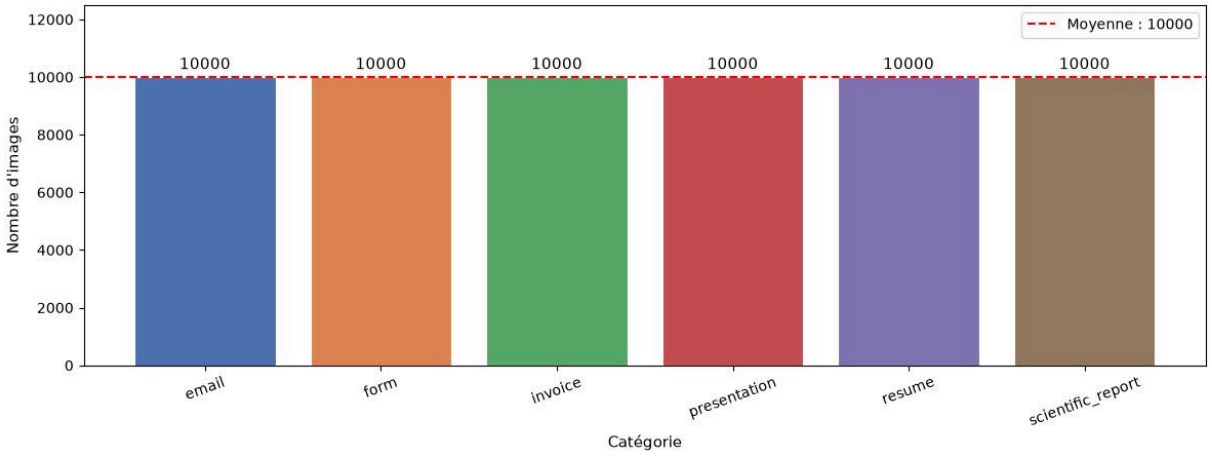
Il est extrait de RVL-CDIP, une base publique de référence de 400 000 documents scannés répartis en 16 classes. Ces documents en anglais viennent d'archives d'entreprises américaines (procès de l'industrie du tabac). Ce sont de vrais scans, souvent abîmés.

Nous avons gardé 6 classes qu'il est possible de rencontrer en entreprise avec 60 000 images.

Pour nos premiers essais nous avons limité notre lot à 30 000 images.

Ci-dessous le graphique représentant la distribution du dataset 2 :

Fig. 1 — Distribution des catégories de documents (RVL-CDIP filtré)

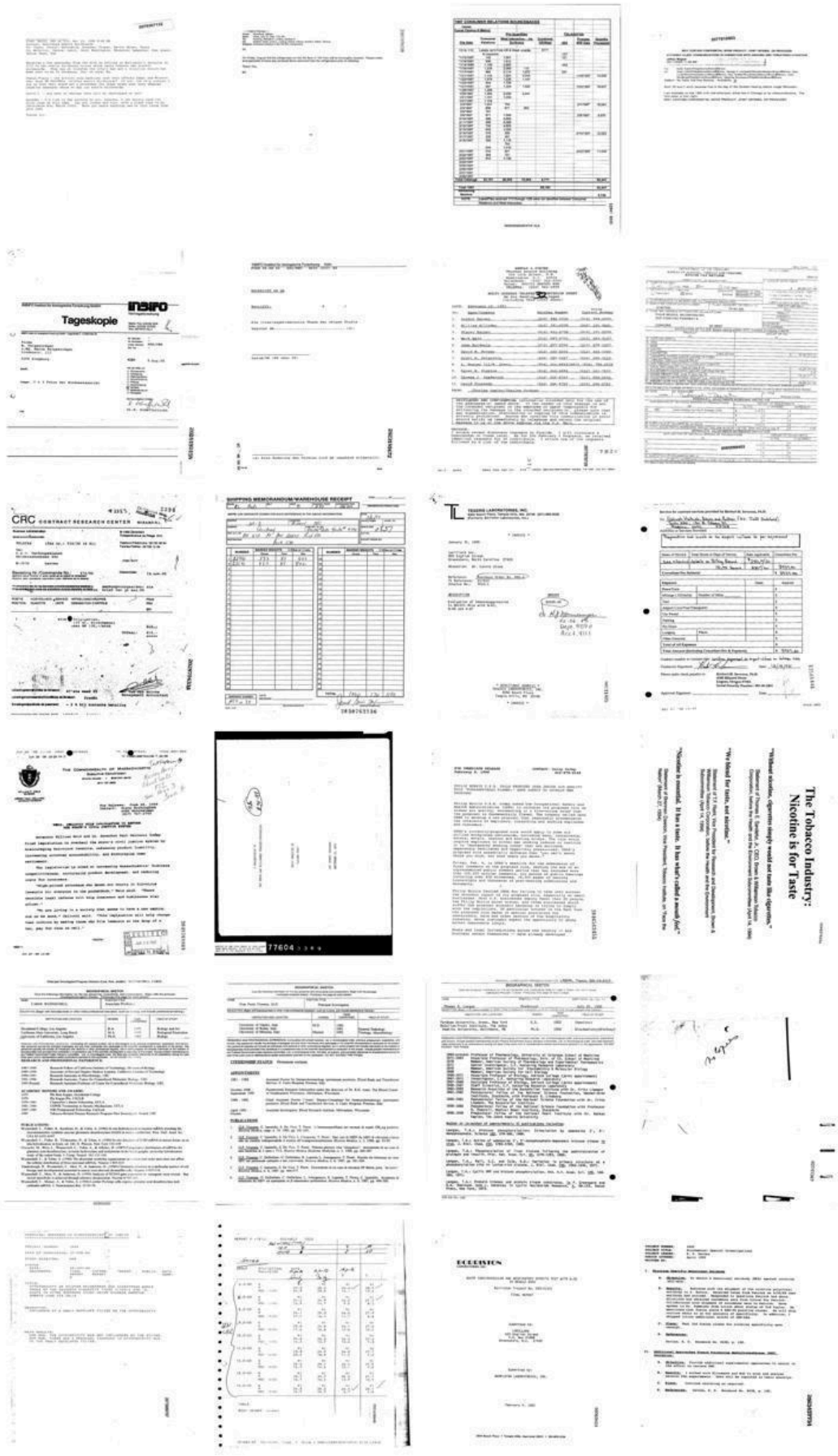


Il n'y a pas de nettoyage à faire sur ce dataset.

Nous avons affiché 4 images au hasard par catégorie et confirmé que ces documents semblent plus proches de ce que nous pourrions avoir dans un cas réel d'entreprise.

Nous observons des documents en format paysage et portrait, beaucoup de champs et de zones de texte sans structure…

Fig. 2 — Exemples visuels par catégorie de document (RVL-CDIP)



Source : 2_exploration/2.2_dataset_2.ipynb

3. Modélisation sur Dataset 1

Nous avons testé deux approches, une méthode sur les features et une méthode sur le texte et les pixels.

3.1 Méthode sur les features

Chaque image est d'abord décrite par 6 mesures simples, calculées en niveaux de gris :

- largeur
- hauteur
- format (largeur / hauteur)
- luminosité (moyenne des pixels)
- contraste (écart-type des pixels)
- part de pixels sombres, qui donne une idée de la densité d'encre

Nous avons comparé 5 classifieurs vus en cours. Leurs hyperparamètres sont réglés par validation croisée sur le jeu d'entraînement (GridSearchCV ou RandomizedSearchCV).

Le jeu de test (33 documents) ne sert qu'à la mesure finale.

Classifieur	Réglage retenu	Accuracy	Erreurs sur 33
SVM	noyau polynomial, C = 10	**75,8 %**	8
KNN	k = 13, distance de Manhattan	69,7 %	10
Random Forest	RandomizedSearchCV (50 essais)	69,7 %	10
XGBoost	GridSearchCV (243 combinaisons)	69,7 %	10
Arbre de décision	profondeur 4 (gini et entropy : même score)	63,6 %	12

Source : 3_dataset_1/3.1_metriques_visuelles.ipynb

Le SVM obtient le meilleur score. Mais les 4 premiers modèles ne sont séparés que par 2 documents : sur 33 documents de test, on ne peut pas vraiment les départager.

Les matrices de confusion montrent que tous les modèles font les mêmes erreurs :

- aucun e-mail n'est mal classé
- des CV sont pris pour des e-mails ou pour des publications scientifiques
- des publications scientifiques sont prises pour des CV

3.2 Méthode sur le texte et les pixels

Avec cette méthode sur le texte et les pixels nous avons testé plusieurs modèles.
Que la représentation soit des pixels bruts, du HOG ou du texte OCR vectorisé, la méthodologie est toujours la même, seul le vecteur de départ change.

Ci-dessous la description de la méthode en prenant HOG comme exemple :

1. Extraction : pour chaque image d'entraînement on calcule son vecteur.

2. Centroïde par classe : pour chaque classe on fait la moyenne des vecteurs de tous les documents d'entraînement de cette classe. On obtient un vecteur typique par classe (3 centroïdes sur ce dataset).

3. Classification : pour une nouvelle image de test, on calcule son vecteur, puis sa similarité cosinus avec chacun des centroïdes.

4. On prédit la classe dont le centroïde à la similarité cosinus la plus élevée c'est à dire l'angle le plus faible entre les deux vecteurs.

Représentations à partir de l'image, sans lire le texte :

- **Pixels bruts** : l'image réduite à 64 × 64 pixels, mise à plat.
- **Pixels centrés** : les mêmes pixels, moins l'image moyenne du jeu d'entraînement, pour enlever le fond blanc commun à toutes les pages.
- **Profils de projection** : pour chaque ligne et chaque colonne de la page, la part de pixels noirs. Cela décrit où se trouve le texte.
- **HOG** (histogramme de gradients orientés) : la page est découpée en petites cases, et dans chaque case on compte l'orientation des contours (horizontaux, verticaux, obliques). Cela résume la forme de la mise en page.
- **Forme** : seulement 2 mesures, le format de la page et la densité d'encre.

Représentations à partir du texte :

Le texte est d'abord lu par OCR (logiciel Tesseract, réglé en anglais), puis transformé en vecteur TF-IDF. Les textes lus ont des longueurs très différentes selon la classe : 92 mots en moyenne pour un e-mail, 280 pour un CV, 461 pour une publication.

- **TF-IDF mots** : chaque mot, ou suite de 2 mots, devient une colonne du vecteur.
- **TF-IDF caractères** : chaque morceau de 3 à 5 caractères devient une colonne. C'est plus robuste aux erreurs de l'OCR, car un mot mal lu garde la plupart de ses morceaux.
- Nous avons aussi testé le retrait des mots vides (the, of, and…). Ils sont présents dans tous les documents, donc ils n'aident pas à les distinguer.

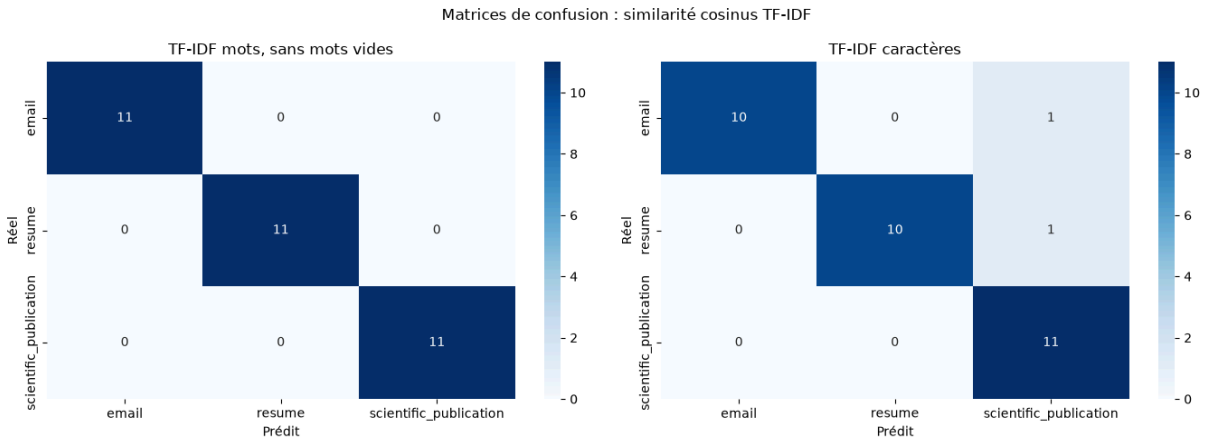
Enfin, nous avons remplacé le cosinus par un vrai classifieur entraîné : un SVM linéaire sur le TF-IDF caractères.

Famille	Représentation	Décision	Accuracy	Erreurs sur 33
Image	Pixels bruts	Cosinus	78,8 %	7
Image	Pixels centrés	Cosinus	69,7 %	10
Image	Profils de projection	Cosinus	81,8 %	6
Image	**HOG**	Cosinus	**90,9 %**	3
Image	Forme (2 mesures)	Cosinus	66,7 %	11
Texte (OCR)	TF-IDF mots	Cosinus	90,9 %	3
Texte (OCR)	**TF-IDF mots, sans mots vides**	Cosinus	**100 %**	0
Texte (OCR)	TF-IDF caractères	Cosinus	93,9 %	2
Texte (OCR)	TF-IDF caractères, sans mots vides	Cosinus	97,0 %	1
Texte (OCR)	TF-IDF caractères	SVM entraîné*	93,9 %	2

* Attention : le SVM n'a pas été évalué sur les mêmes 33 documents de test que les autres modèles (le découpage au hasard est différent). Avec si peu de documents, une erreur vaut 3 points : l'écart entre 93.9 % et 100 % (2 documents) n'est donc pas significatif.

Sources (dossier 3_dataset_1/) : lignes 1 à 5 - 3.2_pixels_cosinus.ipynb, lignes 6 à 9 - 3.2_texte_cosinus.ipynb, ligne 10 - 3.2_texte_svm_3_corpus.ipynb

Ci-dessous la matrice de confusion réalisée sur le meilleur modèle (TF-IDF mots, sans mots vides):



Source : 3_dataset_1/3.2_texte_cosinus.ipynb

Chaque case indique combien de documents d'une vraie classe (ligne) ont été prédits dans chaque classe (colonne). La diagonale représente les documents bien classés. La matrice de confusion montre où se trompe le modèle.

- **TF-IDF mots, sans mots vides** (à gauche) : la diagonale est parfaite, 11 sur 11 dans chaque classe.
- **TF-IDF caractères** (à droite) : 2 erreurs, un e-mail et un CV classés en scientific_publication.

3.3 Constat

Sur ce petit dataset de 3 classes, toutes les méthodes font mieux que le hasard (33 %) : de 63,6 % à 100 %.

C'est la méthode sur le texte est nous permet d'obtenir le meilleur résultat, et avec le retrait des mots vides nous obtenons 100 % (TF-IDF mots sans mots vides).

La méthode sur les pixels bruts, toutes les pages se ressemblent à plus de 99 % car le fond blanc domine. Le score de 78.8 % repose donc sur de très petits écarts.

Mais nous sommes bien conscients à ce stade qu'avec seulement 33 documents de test. Un écart de 1 ou 2 documents n'est pas significatif, et un score de 100 % ne prouve pas que le modèle est parfait.

Ce dataset nous a permis de valider notre méthode. Mais ces documents sont trop peu et trop « propres » pour représenter un vrai cas d'entreprise.

Nous souhaitons donc continuer notre travail sur un autre dataset.

4. Modélisation sur Dataset 2

Nous allons reprendre sur le même principe utilisé pour le dataset 1 mais cette fois notre Dataset 2 qui contient 60 000 images et 6 classes.

Nous commencerons avec la méthode sur les features, puis la comparaison de plusieurs représentations de l'image et du texte.

Nous allons ensuite plus loin avec un réseau de neurones convolutif (partie 4.4), un LLM (partie 4.5) et la combinaison de plusieurs modèles (partie 4.6).

4.1 Méthode sur les features

Voici les résultats des 5 mêmes classifieurs, ré-optimisés sur le premier extrait du dataset 2 : 30 000 images, dont 6 000 pour le test.

Classifieur	Réglage retenu	Accuracy
XGBoost	300 arbres, profondeur 5	**48,2 %**
Random Forest	300 arbres, profondeur 20	47,7 %
SVM	noyau RBF, C = 10	47,3 %
KNN	k = 29, distance de Manhattan	46,1 %
Arbre de décision	profondeur 4	39,6 % (entropy) / 39,9 % (gini)

Source : 4_dataset_2/4.1_metriques_visuelles.ipynb

Les 4 meilleurs modèles arrivent autour de 47-48 %, très loin des 75 % obtenus sur le dataset 1. Les matrices de confusion montrent deux groupes :

- les e-mails et les CV sont assez bien reconnus (80 % et 70 % de bonnes réponses avec le SVM);
- form, invoice, presentation et scientific_report sont très mélangés : moins de la moitié de ces documents sont bien classés.

L'arbre de décision ne s'appuie plus sur la luminosité comme au dataset 1, mais sur le format de la page (42 %) et le contraste (38 %).

Pour améliorer notre test sur les 6 métriques, nous avons demandé à Claude une liste de mesures supplémentaires.

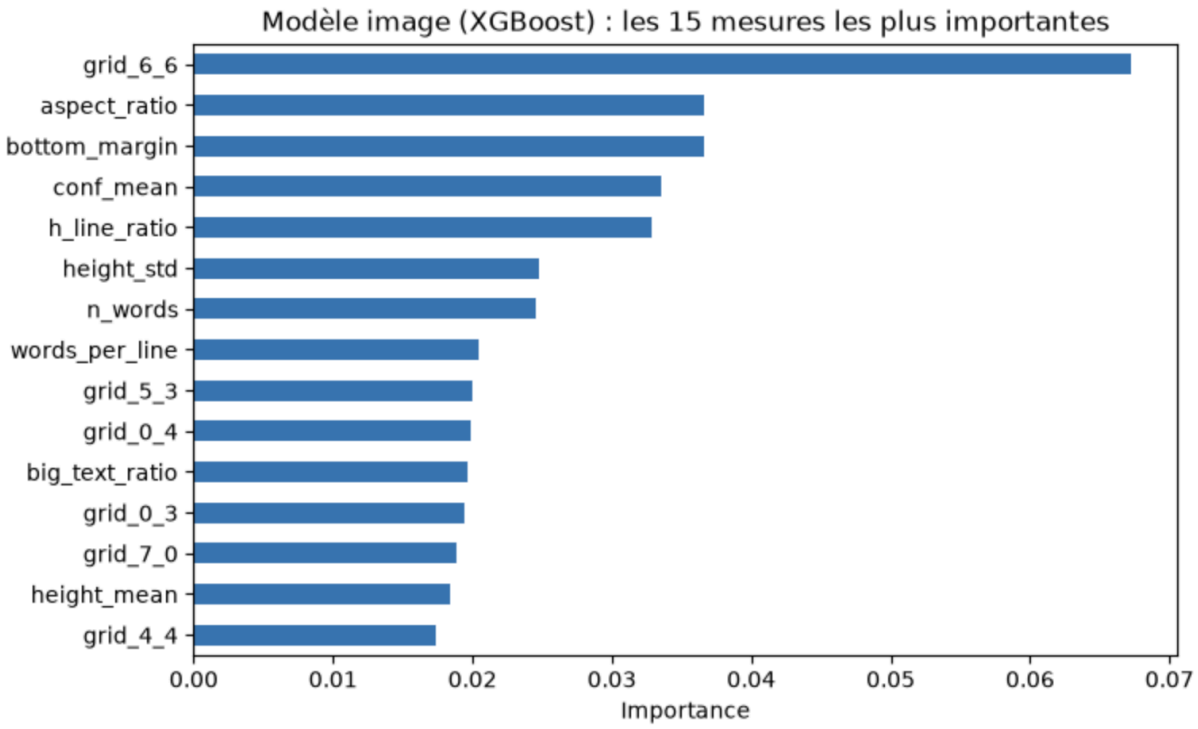
Nous allons relancer notre entraînement avec les 85 mesures ci-dessous :

- **70 mesures sur les pixels** :
  - luminosité, contraste, format de la page et part d'encre ;
  - une grille 8 × 8 qui donne la part d'encre dans chaque case de la page (64 mesures) ;
  - la part de pixels sur des lignes horizontales et verticales, signe d'un tableau ou d'un formulaire.
- **15 mesures de mise en page tirées de l'OCR**, sans les mots eux-mêmes :
  - nombre de mots, de lignes et de blocs ;
  - taille des caractères, marges et alignement ;
  - confiance moyenne de l'OCR, basse quand le texte est difficile à lire (écriture manuscrite, tableau dense, mauvais scan).

Ces 85 mesures sont données à un XGBoost.

Sur les mêmes 2 394 documents de test que le modèle texte, le modèle image atteint **77,7 %**, avec un recall de 80,5 % sur la classe invoice.

Ce graphique montre les 15 mesures qui pèsent le plus dans les décisions du XGBoost.



Source : 4_dataset_2/4.2_modele_image.ipynb

Dans le graphique, les mesures gardent le nom qu'elles ont dans le code. Voici ce qu'elles veulent dire :

Nom dans le graphique	Ce que ça mesure
grid_6_6,grid_5_3, grid_0_4…	la part d'encre dans une case de la grille 8 × 8. Le premier chiffre est la ligne (0 = en haut), le second la colonne (0 = à gauche) : grid_6_6 est donc en bas à droite.
aspect_ratio	le format de la page (largeur divisée par hauteur)
bottom_margin	l'espace vide entre le dernier mot lu et le bas de la page
conf_mean	la confiance moyenne de l'OCR, sur 100
h_line_ratio	la part de pixels sur de longues lignes horizontales (tableaux, champs à remplir)
n_words, words_per_line	le nombre de mots lus, et de mots par ligne
height_mean, height_std	la taille moyenne des caractères, et à quel point elle varie
big_text_ratio	la part de mots bien plus grands que les autres, comme les titres

Pour mieux comprendre, nous avons regardé la moyenne de ces mesures pour chaque classe :

- Les e-mails ont un texte court en haut de la page. Il n'y a presque pas d'encre en bas à droite, et la marge du bas fait 38 % de la page, contre moins de 11 % pour les autres classes.
- Les e-mails et les CV sont tapés proprement : l'OCR y est sûr de lui à 77 sur 100 en moyenne, contre 56 pour les formulaires et 48 pour les factures, souvent remplis à la main ou tamponnés.
- Les formulaires, les factures et les présentations ont environ dix fois plus de lignes horizontales qu'un e-mail, et des caractères de tailles bien plus variées.
- Les CV sont les documents les plus remplis : 346 mots en moyenne, contre 135 pour un e-mail.
- Le format de la page change très peu d'une classe à l'autre (0,76 environ). Le modèle s'en sert quand même, mais c'est la mesure la plus difficile à expliquer.

Source : 4_dataset_2/4.2_modele_image.ipynb

La plus importante est une case de la grille d'encre, en bas à droite de la page.
Viennent ensuite le format de la page, la marge du bas et la confiance moyenne de l'OCR.

Parmi ces 15 mesures, 8 sont calculées sur les pixels et 7 viennent de la mise en page OCR.

4.2 Méthode sur le texte et les pixels

La méthode est la même qu'en partie 3.2.

4.2.1 Méthode sur les pixels

Nous avons utilisé les 30 000 images, dont 6 000 pour le test.

Ci-dessous les résultats du premier essai :

Représentation de l'image	Accuracy
Pixels bruts	44,4 %
Pixels centrés	36,3 %
Profils de projection	43,0 %
HOG	**50,6 %**
Forme (2 mesures)	27,5 %

Source : 4_dataset_2/4.2_pixels_cosinus.ipynb

HOG reste la meilleure représentation de l'image, mais avec 50,6 % elle fait à peine mieux que les 6 métriques globales (48,2 %).

Comme sur le dataset 1, les pixels bruts sont trompeurs, toutes les pages se ressemblent à plus de 99 % au sens du cosinus, car le fond blanc domine.

Nous avons vérifié ce résultat sur l'extrait de 60 000 images et obtenu un score presque identique : 44,7 %.

4.2.2. Méthode sur le texte

L'OCR prend environ une seconde par image. Nous l'avons donc appliqué à un échantillon de 1 800 documents (300 par classe), dont 360 pour le test.

Ci-dessous les résultats :

Représentation du texte	Décision	Accuracy
TF-IDF mots	Cosinus	76,9 %
TF-IDF mots, sans mots vides	Cosinus	77,5 %
TF-IDF caractères	Cosinus	80,6 %
TF-IDF caractères, sans mots vides	Cosinus	79,7 %
Mots + caractères (moyenne pondérée des deux scores)	Cosinus	81,1 %
TF-IDF caractères, plusieurs centroïdes par classe (K-means)	Cosinus	78,9 à 81,9 % selon le nombre de centroïdes

TF-IDF mots, sans mots vides	Régression logistique	79,4 %
TF-IDF mots, sans mots vides	SVM	81,1 %
TF-IDF caractères	Régression logistique	86,1 %
TF-IDF caractères	**SVM**	**86,9 %**

Sources (dossier 4_dataset_2/) : lignes 1 à 6 - 4.2_texte_cosinus.ipynb, lignes 7 à 10 - 4.2_texte_classifieurs.ipynb

Ce que nous montrent ces deux tableaux c'est que le texte fait mieux que l'image (plus de 80 %, contre 50 % au mieux).

Retirer les mots vides dans ce cas ne change presque rien. Le mélange mots + caractères ne gagne que 2 documents sur 360 et le K-means varie de 3 selon le nombre de centroïdes.

Sur exactement les mêmes vecteurs TF-IDF caractères, un SVM entraîné atteint 86,9 % (contre 80,6 % pour le cosinus). C'est notre meilleur résultat.

En vérifiant notre meilleur pipeline, nous avons découvert que l'OCR était réglé en français alors que tous les documents sont en anglais. Le pipeline fonctionnait quand même, mais lisait moins bien certains mots.

La correction (lang="eng") fait passer le SVM sur TF-IDF caractères de 83,9 % à 86,9 %, soit 11 documents de plus sur 360, sur les mêmes documents et avec le même réglage. L'OCR lit aussi plus de mots : 135 par document en moyenne, contre 123. Tous les résultats de ce rapport utilisent l'OCR corrigé.

Source : 4_dataset_2/4.2_texte_classifieurs.ipynb

Le SVM sur TF-IDF caractères est notre meilleur modèle.

Nous l'avons ré-entraîné sur l'extrait de 60 000 images, en appliquant l'OCR à 12 000 documents (2 000 par classe). Après retrait des documents où l'OCR ne lit rien, il reste 9 572 documents d'entraînement et 2 394 de test.

Voici le même pipeline appliqué aux 3 corpus :

Corpus	Classes	Documents de test	Accuracy
Dataset 1	3	33	93,9 %
Dataset 2, 30 000 images	6	360	84,4 %
Dataset 2, 60 000 images	6	2 394	**86,7 %**

Sources : 3_dataset_1/3.2_texte_svm_3_corpus.ipynb ; modèle de référence : 4_dataset_2/4.2_texte_svm_60k.ipynb

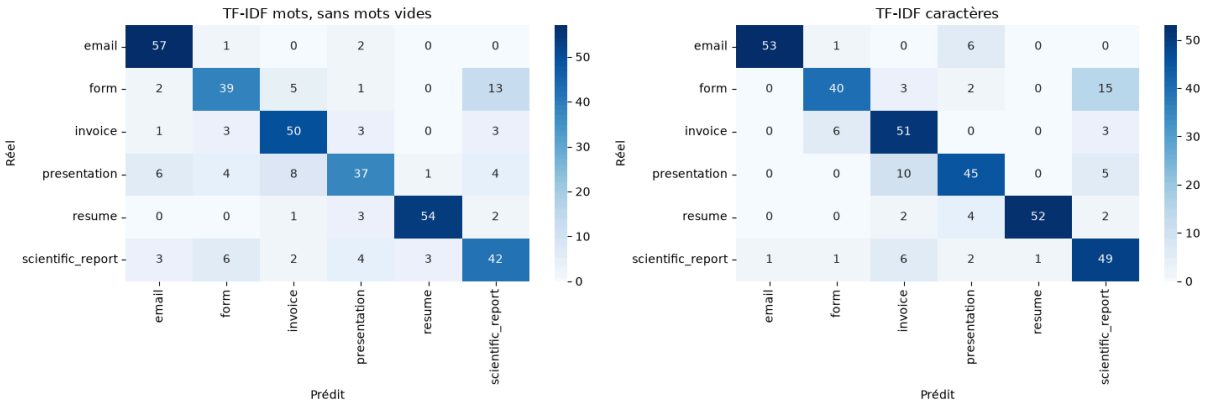
Ce SVM à 86,7 % est notre modèle texte de référence pour la suite.

Nous avons maintenant deux modèles mesurés sur les mêmes documents :

- un modèle texte => 86,7 %
- un modèle image => 77,7 %

Suite à ce résultat nous avons poussé l'analyse puisque l'accuracy donne un score global, mais ne dit pas quelles classes sont confondues entre elles.

Voici les matrices des deux modèles texte par cosinus réalisées sur 60 documents de test par classe. TF-IDF mots à gauche, TF-IDF caractères à droite.

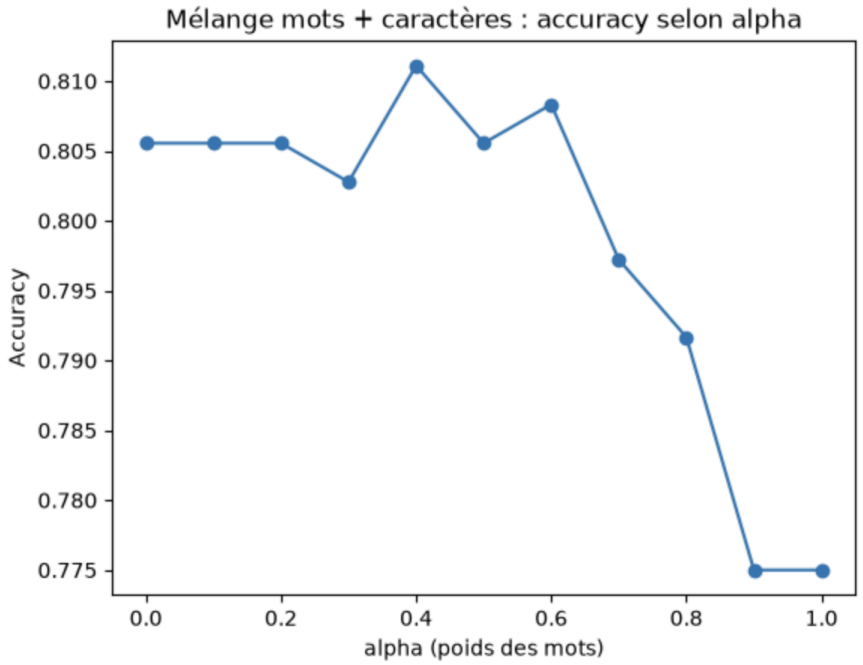


Source : 4_dataset_2/4.2_texte_cosinus.ipynb

Sur les deux matrices, l'erreur la plus fréquente est la même : des formulaires (form) classés en rapports scientifiques (scientific_report). Cela arrive à 13 formulaires sur 60 avec les mots, et à 15 avec les caractères.

Nous pensons que dans ces archives, beaucoup de formulaires servent à décrire des travaux scientifiques (fiches de laboratoire, fiches d'articles). Leur vocabulaire ressemble donc à celui d'un rapport, et un modèle qui ne lit que le texte les confond.

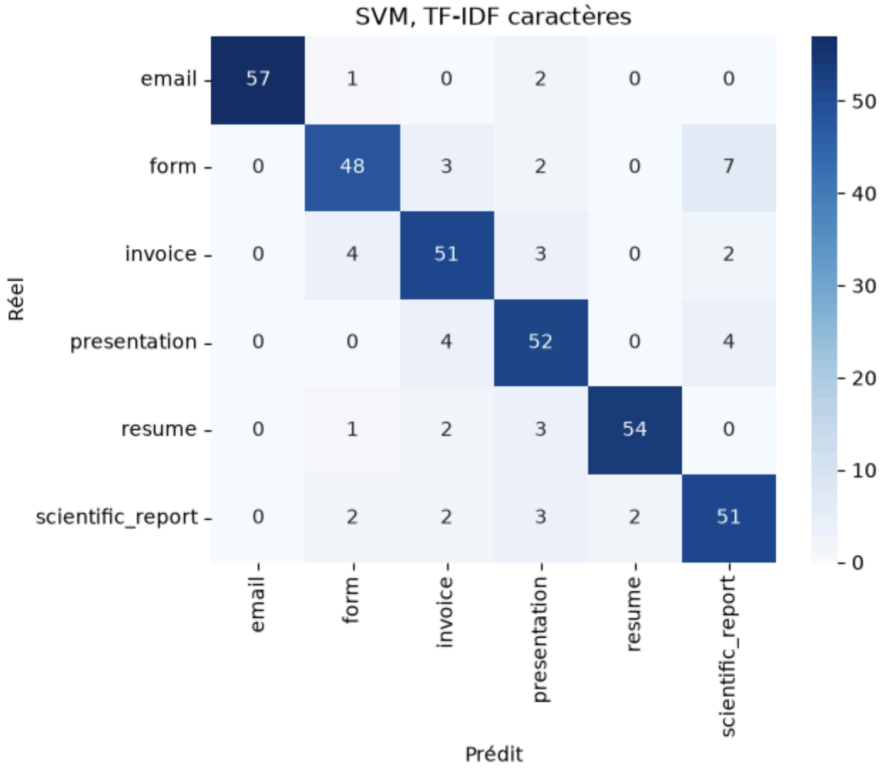
Nous nous demandons s'il est possible d'améliorer le score en combinant les deux représentations du texte ?
Nous avons fait une moyenne pondérée des deux scores, alpha × score mots + (1 − alpha) × score caractères, en faisant varier alpha de 0 (caractères seuls) à 1 (mots seuls).



Source : 4_dataset_2/4.2_texte_cosinus.ipynb

Attention, on parle ici de la similarité cosinus, sans SVM. Le meilleur réglage donne 81,1 %, contre 80,6 % pour les caractères seuls, soit 2 documents de plus sur 360. Et encore, alpha a été choisi en regardant directement le test : ce 81,1 % est donc plutôt optimiste. Mots et caractères lisent le même texte et font les mêmes erreurs : les combiner n'apporte rien.

Voici enfin la matrice du meilleur modèle de cet extrait, le SVM sur TF-IDF caractères (86,9 %).

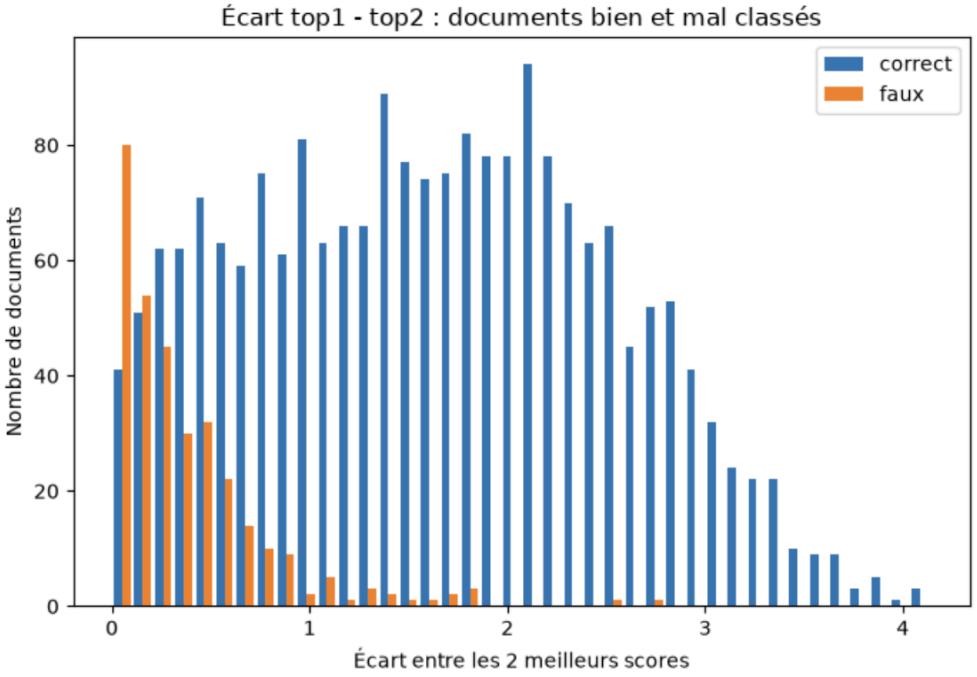


Source : 4_dataset_2/4.2_texte_classifieurs.ipynb

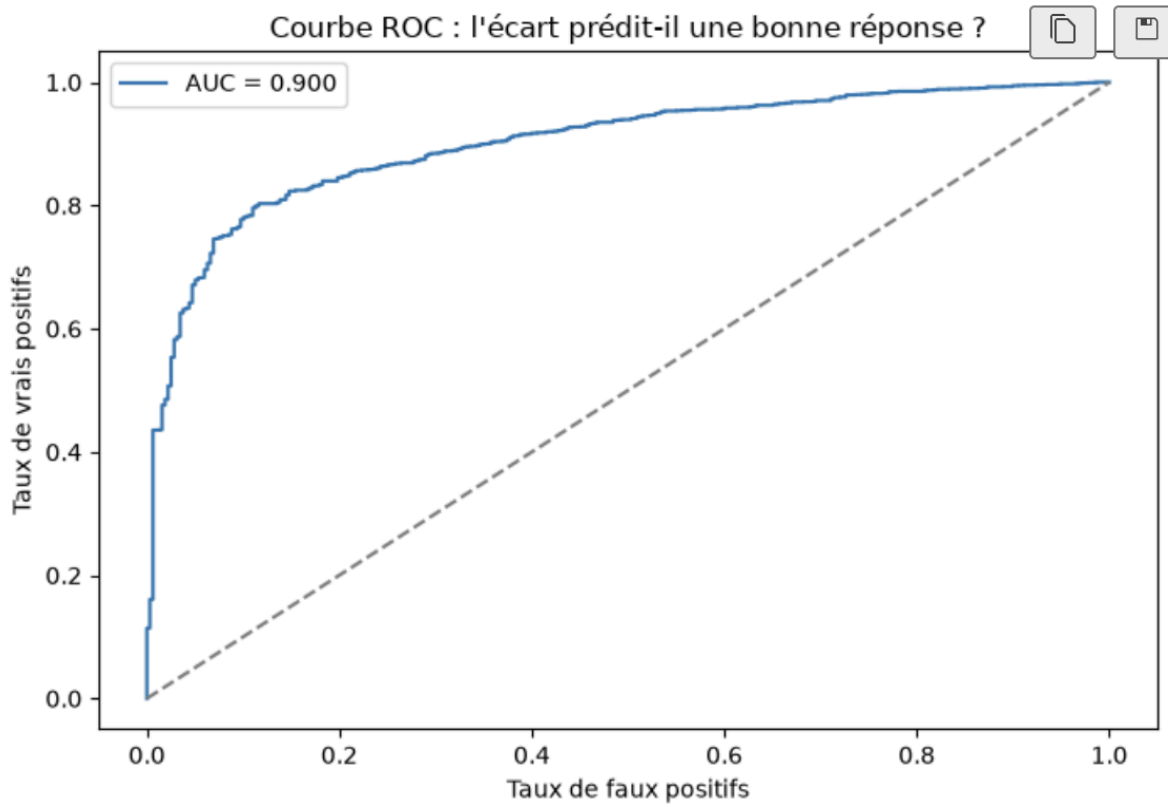
Presque toutes les classes progressent, mais des formulaires sont encore classés en rapports scientifiques. Les autres erreurs se répartissent entre form et invoice : 7 documents, dans les deux sens ainsi que des presentations classées en invoice ou en scientific_report : 4 documents chacun.

Nous pensons que ce n'est donc pas seulement un problème de modèle, certains documents sont réellement ambigus quand on ne lit que leur texte.

Nous nous sommes ensuite demandé si le modèle sait quand il se trompe. Pour chaque document, le SVM donne un score à chacune des 6 classes. Nous regardons l'écart entre les deux meilleurs scores : un petit écart signifie que le modèle hésite entre deux classes. Cette analyse porte sur le modèle de référence (60 000 images, 2 394 documents de test).



Ce premier graphique montre la répartition de cet écart, pour les documents bien classés et mal classés. Les erreurs sont presque toutes du côté des petits écarts => Quand le modèle se trompe, c'est qu'il hésite.



Ce second graphique utilise cet écart pour prédire si la réponse du modèle est juste. La courbe ROC montre le résultat pour tous les seuils possibles.
L'aire sous la courbe (AUC) vaut 0,900 : 1 correspondrait à une prédiction parfaite, 0,5 au hasard.

Source : 4_dataset_2/4.2_texte_svm_60k.ipynb

Concrètement, sur le tiers des documents où l'écart est le plus faible (800 documents), l'accuracy tombe à 63,7 %, contre 86,7 % sur l'ensemble. Puisque le modèle sait quand il hésite, nous pouvons concentrer nos efforts sur ces documents plutôt que de tout refaire (partie 4.6).

4.3 Méthode sur le CNN – deep learning

Afin de savoir si les 85 mesures choisies à la main sont la meilleure façon d'extraire un signal de l'image, nous avons testé deux stratégies du cours de Transfer Learning, sur les mêmes documents que le modèle texte de référence.

Nous comparerons les résultats à notre modèle SVM texte qui a obtenu 86,7 % est qui est notre référence.

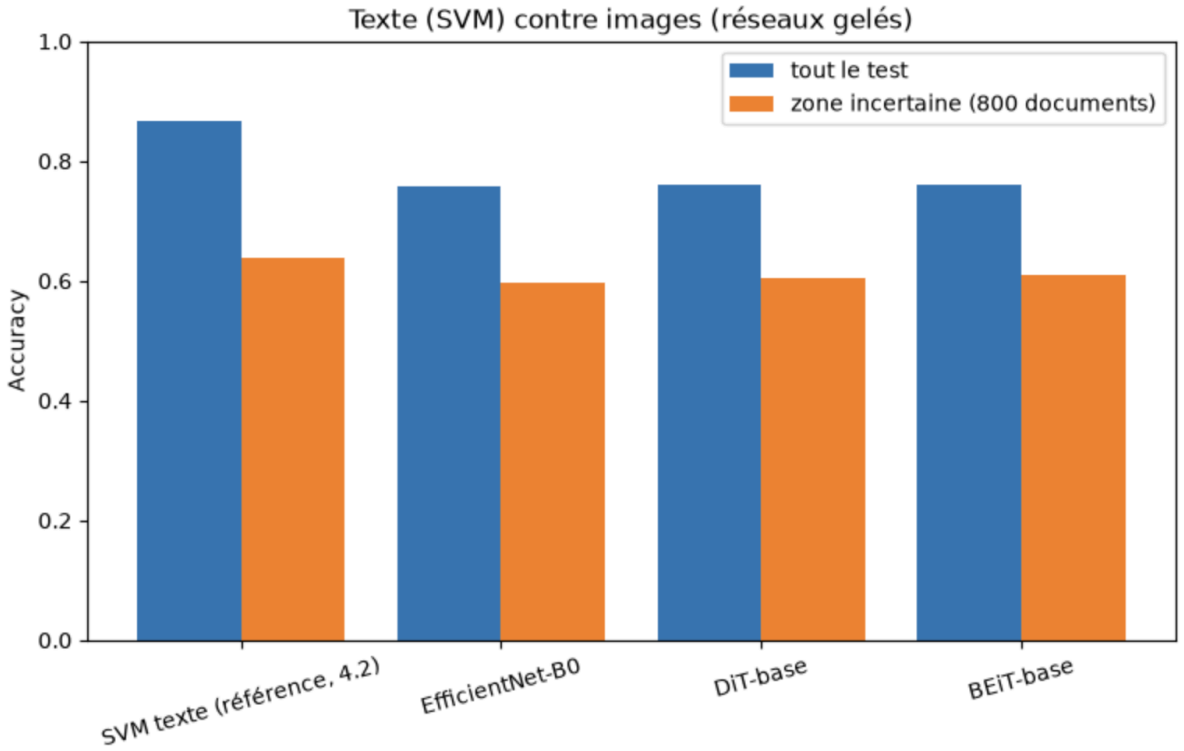
4.3.1 Feature Extraction : 3 réseaux pré-entraînés et gelés

Le réseau pré-entraîné est gelé, aucun de ses poids ne change. On garde seulement le vecteur pour chaque image, et on entraîne une régression logistique dessus.

Réseau	Pré-entraîné sur	Accuracy	Accuracy zone incertaine
SVM texte (référence)	—	86,7 %	63,7 %

EfficientNet-B0 (CNN)	ImageNet (photos)	75,8 %	59,6 %
DiT-base (Transformer)	IIT-CDIP (documents, sans étiquettes)	76,1 %	60,5 %
BEiT-base (Transformer)	ImageNet-22k (photos, sans étiquettes)	76,0 %	61,0 %

Nous avions d'abord testé un DiT déjà entraîné à classer RVL-CDIP. Sans entraînement de notre part, il atteint 98,1 %. Ce score n'est pas utilisable : ce modèle a appris sur la partie de la base d'où viennent nos images, avec leurs étiquettes. Nous l'avons écarté et gardé 3 réseaux qui n'ont jamais vu nos étiquettes.



Source : 4_dataset_2/4.4_feature_extraction.ipynb

Les 3 réseaux sont presque égaux, autour de 76 %. Nous ne ferons pas le Fine-Tuning sur les 3 modèles, nous choisissons de le faire seulement sur le EfficientNet-B0.

4.3.2 Fine-Tuning total: un CNN EfficientNet-B0

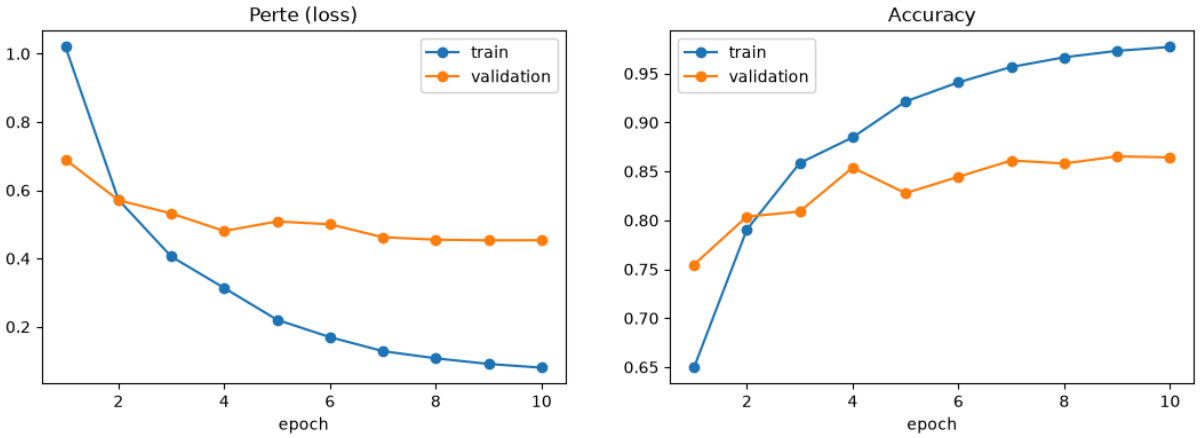
Ces résultats encourageants justifient un vrai entraînement. Nous avons choisi EfficientNet-B0, le plus léger des trois réseaux.

Nous appliquons la stratégie Fine-Tuning total du cours. La dernière couche du réseau est remplacée par une couche à 6 classes, puis tout le réseau est ré-entraîné sur nos images.

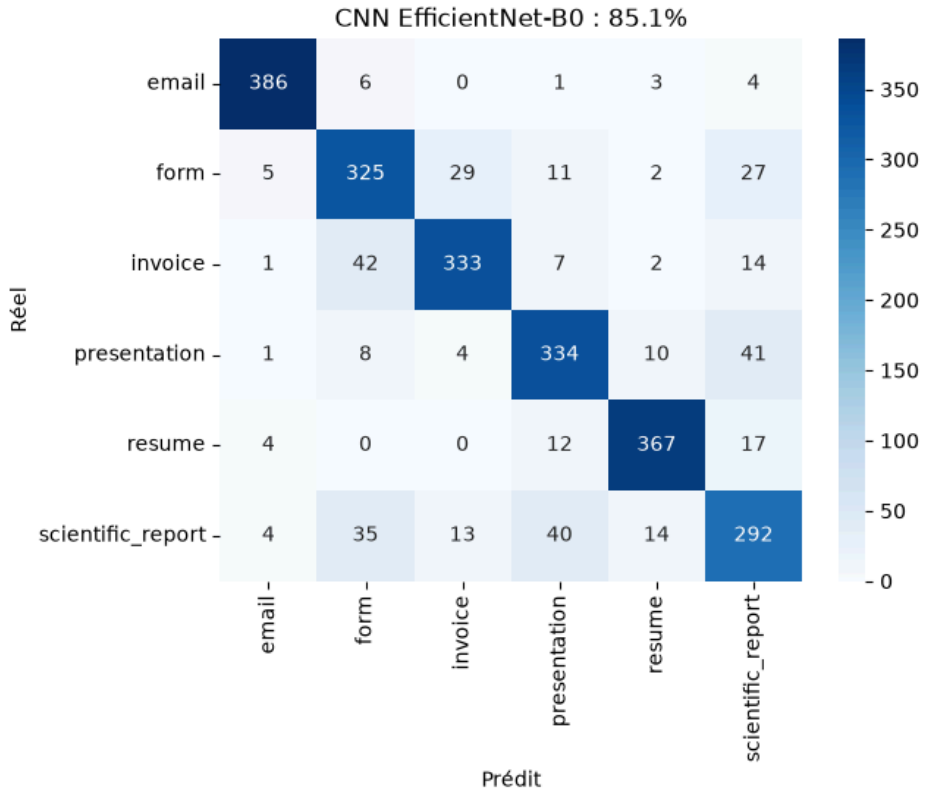
Nos réglages :

- Images en 384 × 384 pixels, pour que le texte reste lisible.
- Augmentation légère : petite rotation, luminosité et contraste. Pas de retournement, qui n'a pas de sens pour un document.

- 10 epochs au maximum, avec arrêt anticipé (early stopping) sur une validation de 10 % du train.



Le train continue de progresser (98 % d'accuracy) alors que la validation plafonne autour de 86 % dès l'epoch 4 : c'est un début de surapprentissage. Nous gardons les poids de la meilleure epoch sur la validation.



Source : 4_dataset_2/4.4_cnn.ipynb

Le CNN EfficientNet-B0 fine tuné atteint **85,1 %** sur le test, à peine moins que le SVM texte (86,7 %) mais plus que notre modèle sur les 85 mesures (77,7 %).

Nous observons qu'il reconnaît mieux les e-mails, formulaires et présentations (96,5 %, 81,5 % et 83,9 % de recall, contre 94,2 %, 77,7 % et 80,9 % pour le SVM texte).
Il est moins bon sur les factures, CV et rapports scientifiques.

Mais il classe correctement 194 des 319 documents où le modèle SVM texte s'est trompé.
C'est ce qui rend la combinaison intéressante et nous le développerons dans le partie 4.6.3.

4.4 Méthode LLM

Nous avons voulu tester en envoyant directement l'image à un LLM local (gemma4, via Ollama), avec un prompt qui liste les classes possibles.

Chaque appel prend plusieurs secondes, nous avons donc testé sur un petit échantillon de 10 images par classe :

- **Dataset 1** (3 classes, 30 images) : **93,3 %** ;
- **Dataset 2**, premier extrait (6 classes, 60 images) : **73,3 %**.

Source : 4_dataset_2/4.5_llm.ipynb

Sur le dataset simple, le LLM est très bon sans aucun travail de préparation. Sur 6 classes, il fait les mêmes erreurs que nos autres modèles, en plus marqué : 7 présentations sur 10 sont prises pour des rapports scientifiques, et 5 factures sur 10 pour des formulaires.

Avec 73,3 %, il reste loin du SVM texte (86,9 % sur le même extrait) et de plus chaque prédiction prend plusieurs secondes, contre quelques millisecondes pour nos modèles.

Nous ne poussons pas plus loin l'analyse avec une méthode LLM mais préférons améliorer notre modèle SVM texte.

4.5 Améliorer le modèle SVM sur TF-IDF caractères

Nous allons explorer deux pistes :

1. retravailler seulement les documents où le modèle hésite (4.6.1) ;
2. combiner des modèles qui ne regardent pas la même chose : texte, image, CNN (4.6.2 et 4.6.3).

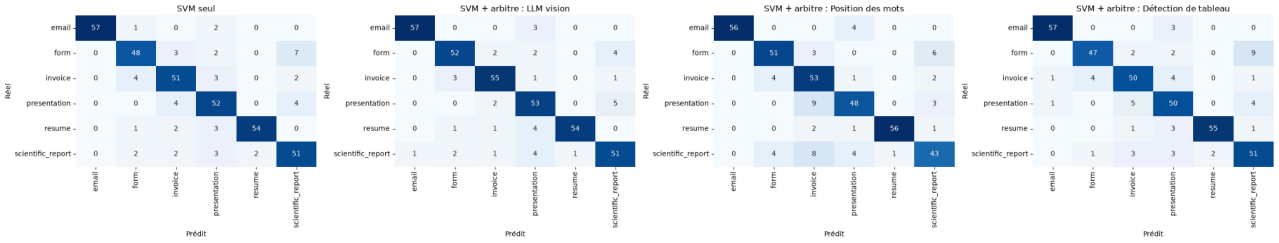
4.5.1 Arbitrer seulement les cas les plus durs (top1 vs top2)

Le modèle hésite presque toujours entre deux classes. Pour les documents où l'écart entre ses deux meilleurs scores est faible, nous gardons ces deux candidats et demandons à un second signal de trancher. Les autres documents ne changent pas.

Nous avons testé trois arbitres sur le premier extrait, avec le SVM sur TF-IDF caractères (86,9 %) comme référence. Les trois arbitres traitent les mêmes 67 documents incertains, sur lesquels le SVM n'a que 56,7 % de bonnes réponses.

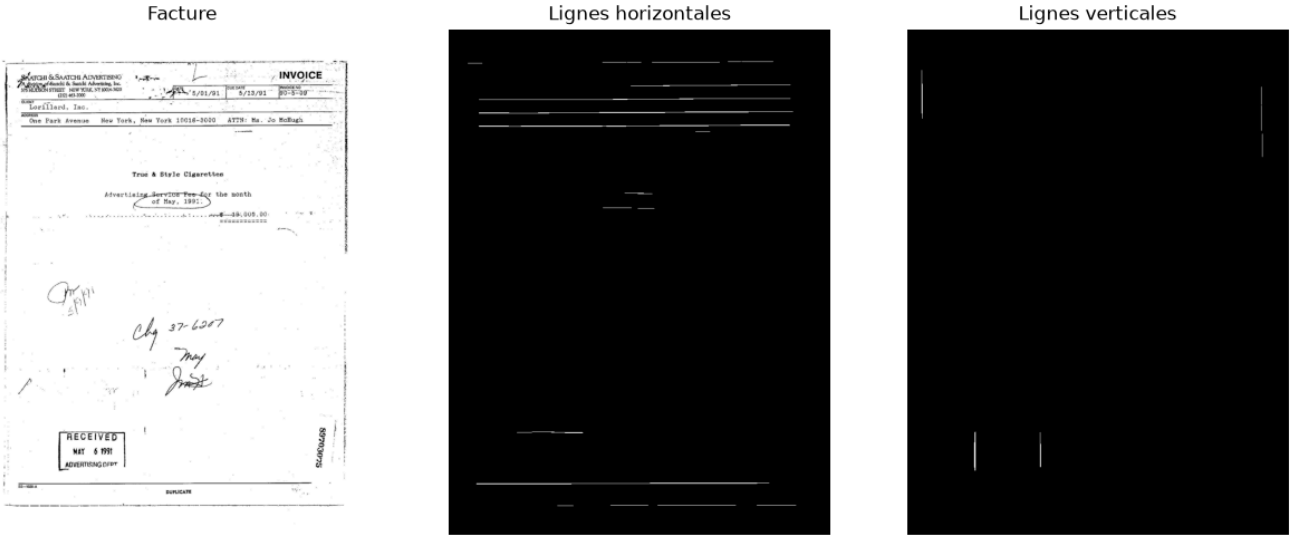
Arbitre	Ce qu'il regarde	Corrigés	Cassés	Gain net	Accuracy globale
Aucun (SVM seul)	—	—	—	—	86,9 %

LLM vision	l'image, avec seulement les 2 classes candidates dans le prompt	14	5	**+ 9**	**89,4 %**
Position des mots	6 mesures tirées des boîtes des mots lus par l'OCR	11	17	− 6	85,3 %
Détection de tableau	les lignes horizontales et verticales de l'image	13	16	− 3	86,1 %



Source : 4_dataset_2/4.6.1_arbitrage_top1_top2.ipynb

Seul le LLM améliore le résultat, parce qu'il « voit » vraiment la structure du document (tableaux, cases, en-têtes). Les deux autres arbitres n'en mesurent qu'une approximation : ils corrigent quelques documents, mais en cassent davantage.



Source : 4_dataset_2/4.6.1_arbitrage_top1_top2.ipynb

Cette illustration montre ce que « détection de tableau » veut dire. L'image passe en noir et blanc, puis une opération de morphologie (OpenCV) ne garde que les traits longs et continus.
Ces modifications ne suffisent pas à séparer des classes proches.

De plus, faire un appel au LLM de plusieurs secondes par document incertain n'est pas optimisé si on imagine que notre modèle passe à l'échelle de millions de documents.

A ce stade nous identifions également une problématique sur les documents incertains mis en hésitation.

Source : 4_dataset_2/4.6.1_meta_modele.ipynb

Nous avons alors essayé autre chose sur ces documents incertains. Un méta-modèle (régression logistique) voit les 6 scores du SVM, l'écart entre les deux meilleurs, et 10 mesures d'image : 4 de détection de tableau et 6 de mise en page.

Il ne remplace la prédiction du SVM que s'il est très confiant, au-dessus d'un seuil choisi sur un jeu de validation.

Les documents incertains sont découpés en entraînement (2 543), validation (848) et test (848). Le test ne sert qu'une fois, à la fin. Sur ce test, le méta-modèle corrige 8 documents sans en casser aucun.

Le test de McNemar confirme que ce gain n'est pas dû au hasard (p = 0,0078). Ce test compare deux modèles sur les mêmes documents, en regardant seulement ceux où ils ne sont pas d'accord.

Ce gain est quand même fragile. Avec un seul changement raisonnable, standardiser les variables avant la régression logistique, il tombe à + 4 documents et n'est plus significatif (p = 0,125). L'effet reste petit.

Source : 4_dataset_2/4.6.1_meta_modele.ipynb

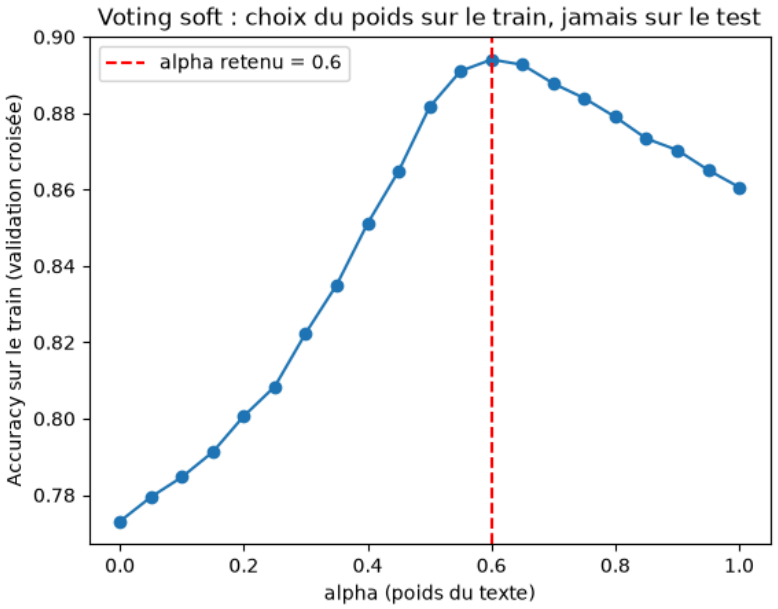
4.5.2 Combiner le texte et l'image : Voting et Stacking

Nous combinons le modèle texte (SVM, 86,7 %) et le modèle image (XGBoost sur 85 mesures, 77,7 %) avec les deux méthodes d'ensemble du cours :

- **Voting soft** (`VotingClassifier`) : moyenne pondérée de leurs probabilités, avec un poids alpha pour le texte et 1 − alpha pour l'image ;
- **Stacking** (`StackingClassifier`) : un méta-modèle (régression logistique) apprend à combiner les probabilités des deux modèles.

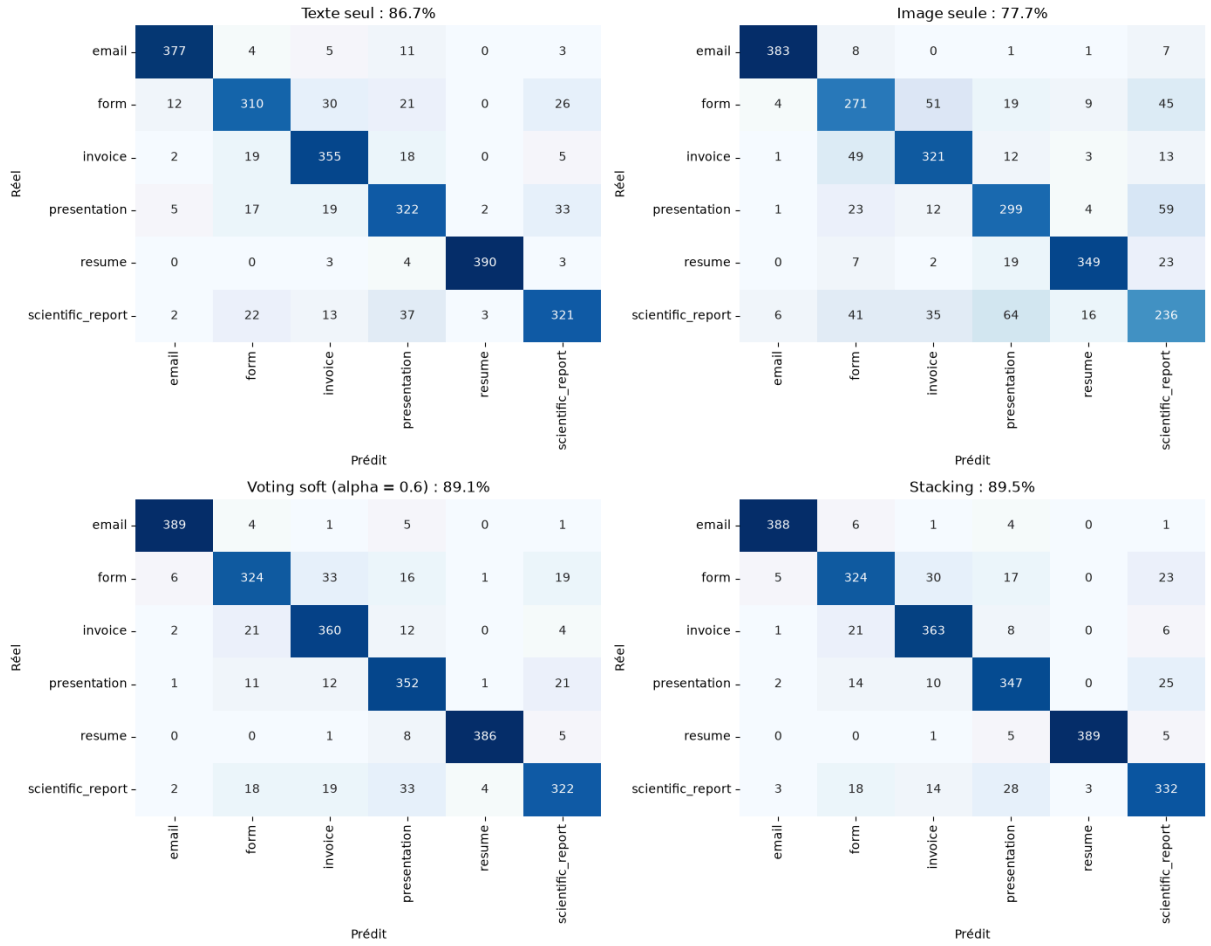
Le SVM linéaire ne donne pas directement de probabilités : `CalibratedClassifierCV` les calcule à partir de ses scores.

Comme dans le cours, le méta-modèle apprend sur des probabilités obtenues par validation croisée sur le train : chaque document est prédit par des modèles qui ne l'ont pas vu. Le poids alpha du Voting est lui aussi choisi sur le train (graphique ci-dessous). Le test ne sert qu'une fois, pour la mesure finale.



Le meilleur poids est alpha = 0,6 : le texte compte pour 60 %, l'image pour 40 %.

Configuration	Accuracy	Recall invoice
Texte seul	86,7 %	89,0 %
Image seule	77,7 %	80,5 %
Voting soft (alpha = 0,6)	89,1 %	90,2 %
Stacking	**89,5 %**	**91,0 %**



Source : 4_dataset_2/4.6.2_stacking_texte_image.ipynb

La combinaison gagne près de 3 points sur le texte seul. Le gain n'est pas le même partout : il se concentre sur les classes faibles du texte, les formulaires et les présentations.

Nous avons refait le calcul avec 5 découpages différents de la validation croisée. Le Stacking bat le Voting soft dans les 5 cas.

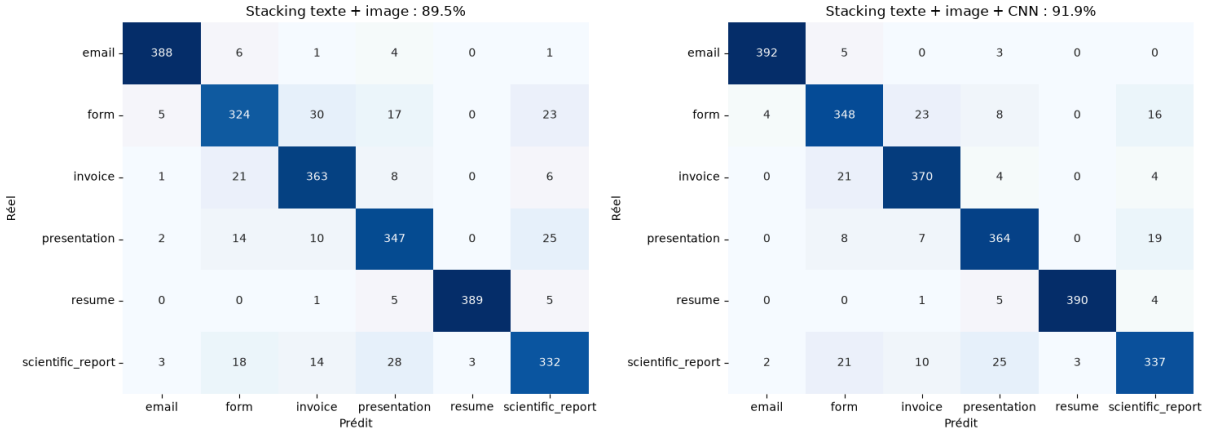
4.5.3 Stacking à 3 modèles : texte + image + CNN

Nous ajoutons le CNN (Partie 4.4) comme troisième modèle. Les trois modèles regardent des choses différentes :

	Modèle texte	Modèle image	CNN
Entrée	texte OCR → TF-IDF caractères	85 mesures choisies à la main	image brute (384 × 384)
Classifieur	SVM	XGBoost	EfficientNet-B0
Accuracy seul	86,7 %	77,7 %	85,1 %

Le Stacking demande, pour chaque document du train, les probabilités de chaque modèle calculées par validation croisée. Chaque document est prédit par un modèle qui ne l'a pas vu pendant son entraînement. Pour le texte et l'image, `StackingClassifier` s'en charge tout seul. Le CNN, lui, s'entraîne avec PyTorch : nous avons donc calculé ses probabilités de la même façon, mais séparément, puis entraîné la régression logistique sur les probabilités des trois modèles.

Configuration	Accuracy	Recall invoice
Stacking texte + image (4.6.2)	89,5 %	91,0 %
Stacking texte + CNN	91,6 %	91,7 %
Stacking texte + image + CNN	**91,9 %**	**92,7 %**
Stacking à 3, méta-modèle GradientBoosting	91,8 %	93,0 %
Voting soft à 3 (poids 0,5 / 0,2 / 0,3)	91,5 %	92,7 %



Source : 4_dataset_2/4.6.3_stacking_texte_image_cnn.ipynb

C'est le meilleur résultat du projet : 91,9 %, soit + 5,2 points sur le texte seul.

- Ajouter le CNN fait passer les erreurs de 251 à 193 documents.
- Les plus gros progrès portent sur les formulaires et les présentations.
- Le modèle image fait de moins en moins de différence une fois le CNN présent : + 0,3 point seulement par rapport au stacking texte + CNN.

Sur 5 découpages différents de la validation croisée, ajouter le CNN fait gagner entre 2,4 et 2,7 points, soit + 2,5 points en moyenne.

Les 3 modèles et le méta-modèle, entraînés sur le dataset 2, ont été appliqués sans aucun ré-entraînement aux 165 documents du dataset 1. Seules les classes email et resume existent dans les deux datasets. Sur ces 110 documents, le stacking atteint **100 %**, contre 99,1 % pour le texte seul. Les 55 publications scientifiques, qui n'ont pas de classe identique, sont presque toutes rangées en scientific_report (53 sur 55), la classe la plus proche. Ce test reste partiel (2 classes en commun).

4.6 Constat

- **Le texte est le signal le plus fort.** Sur le dataset 2, la meilleure approche purement visuelle atteint 50,6 % (HOG) et le texte 86,9 % (SVM) : 36 points d'écart.
- **La règle de décision compte autant que la représentation.** Sur les mêmes vecteurs TF-IDF caractères, remplacer le cosinus par un SVM entraîné fait passer l'accuracy de 80,6 % à 86,9 %.
- **Combiner des modèles qui regardent des choses différentes paie.** Combiner deux lectures du même texte (mots + caractères) n'apporte rien. Combiner le texte, l'image et le CNN fait passer de 86,7 % à 89,5 %, puis à **91,9 %**.
- **Le LLM n'est pas compétitif seul, mais il est utile en arbitre.** Il est trop lent et trop imprécis sur 6 classes (73,3 %). En revanche, sur quelques documents difficiles, c'est le seul arbitre qui améliore le résultat (+ 9).

5. Explicabilité du modèle CNN

Nous n'allons pas traité l'explicabilité de tous nos modèles mais nous choisissons de réaliser notre partie explicabilité sur le modèle CNN.

Le modèle SVM texte donne un poids à chaque morceau de mot et le XGBoost indique quelles mesures comptent le plus (partie 4.2). Le CNN, lui, nous paraît moins clair, il classe bien (85,1 %), mais ses 4 millions de paramètres ne disent pas sur quoi il s'appuie.

Ce modèle obtient un bon résultat (partie 4.6.3), nous voulons vérifier qu'il regarde bien le contenu du document, et non un défaut de numérisation (marge noire, bord de page). Et voir si nous pouvons expliquer pourquoi il confond certaines classes, en particulier invoice et form.

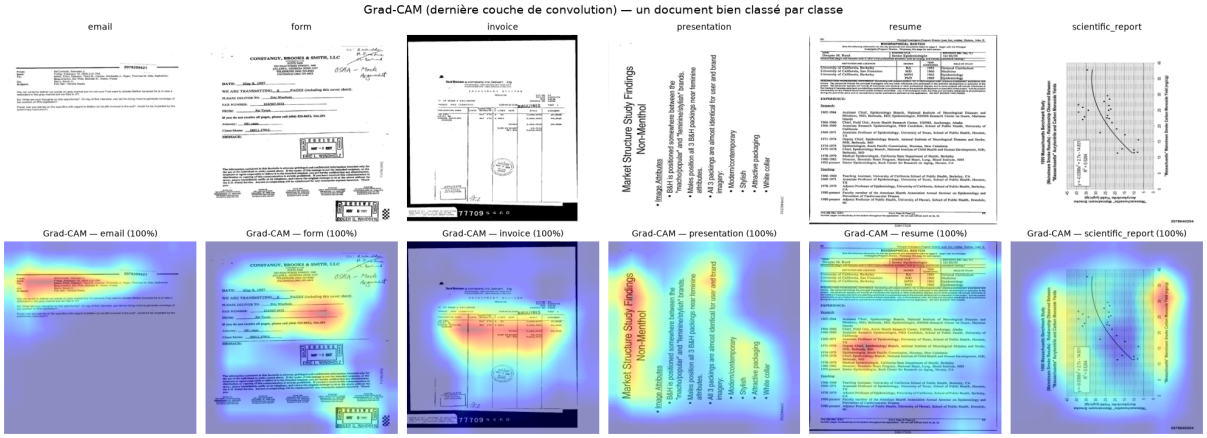
Nous utilisons les outils du cours : Grad-CAM, la visualisation des feature maps et SHAP.

Le modèle étudié est exactement celui de la partie 4.4, rien n'est ré-entraîné.

5.1 Grad-CAM

Grad-CAM produit une carte de chaleur superposée à l'image. Les zones rouges sont celles qui ont le plus fait monter le score de la classe prédite. Pour la calculer, on mesure l'effet de chaque zone de la dernière couche de convolution sur ce score, à l'aide des gradients.

Pour chaque classe, nous prenons le document de test que le CNN classe correctement avec la plus grande confiance.



Source : 5_explicabilite/5_explicabilite_cnn.ipynb

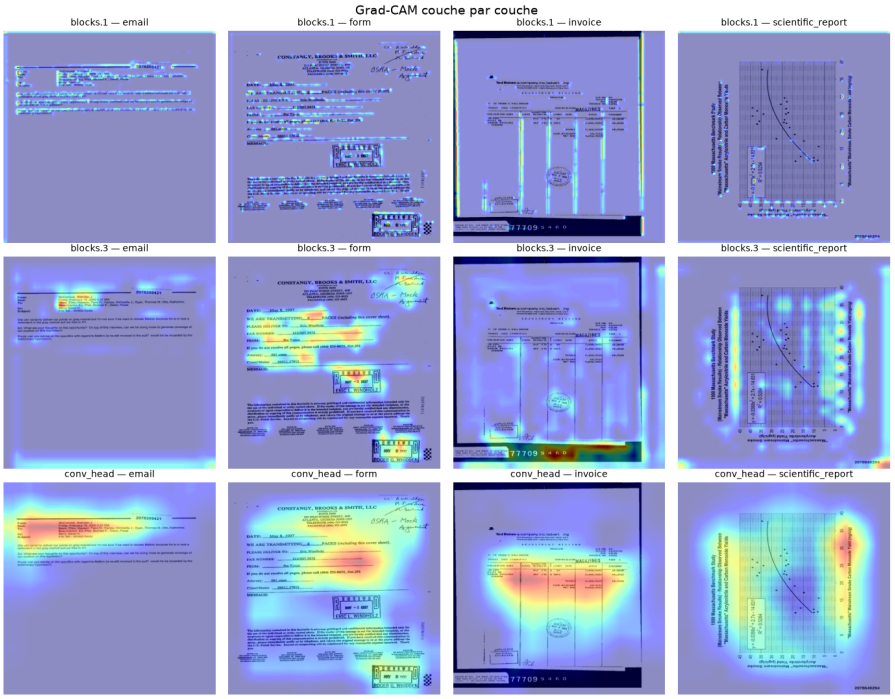
Sur chaque document, la zone rouge tombe sur le bloc de mise en page typique de la classe :

- **email** : le bloc d'en-tête (From, Sent, To, Subject), pas le corps du message ;
- **form** : les lignes de champs à remplir, au centre de la page ;
- **invoice** : les lignes du tableau de facturation ;
- **presentation** : le titre et la liste à puces, bien que la page soit scannée à 90° ;
- **resume** : le tableau de formation en haut de la page ;
- **scientific_report** : le graphique et les légendes de ses axes.

Le modèle ne s'appuie pas sur un défaut de numérisation et il repère un en-tête, un tableau ou une liste, sans lire ce qui est écrit.

Le cours montre qu'un CNN détecte des motifs de plus en plus complexes au fil des couches. Nous le vérifions de deux façons.

D'abord, nous calculons Grad-CAM sur trois couches, du début à la fin du réseau.

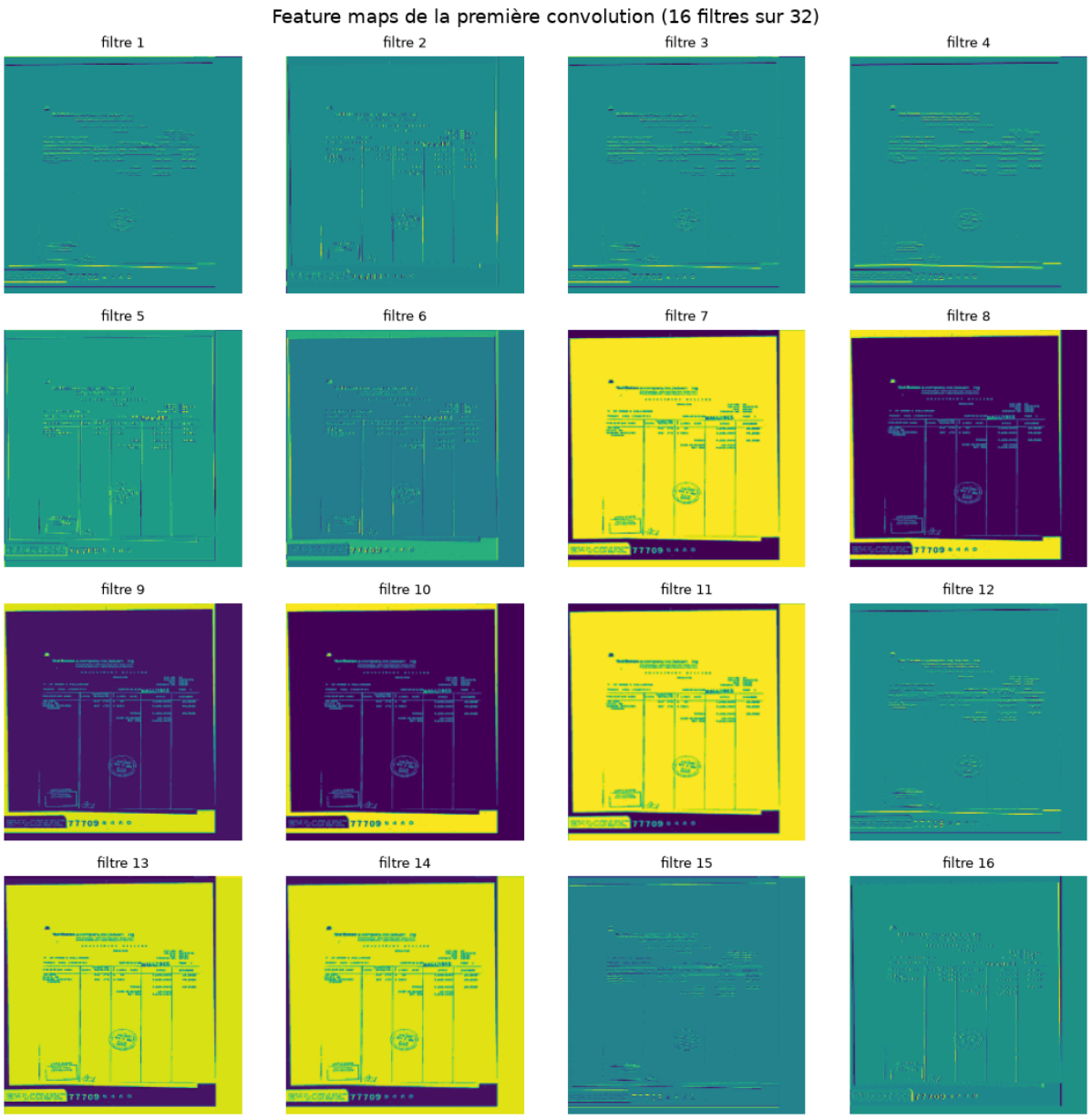


Au début du réseau les traits sont fins. On reconnaît les lignes de texte, les filets verticaux du tableau, le quadrillage du graphique.

Au milieu du réseau, ce sont des objets de mise en page comme les valeurs écrites dans les champs, le tampon ou la bande de chiffres en bas de la facture.

A la dernière couche ce sont de grandes zones, chacune couvrant un bloc entier du document.

Ensuite, nous affichons les feature maps de la toute première convolution : la sortie brute de 16 de ses 32 filtres, pour la facture.

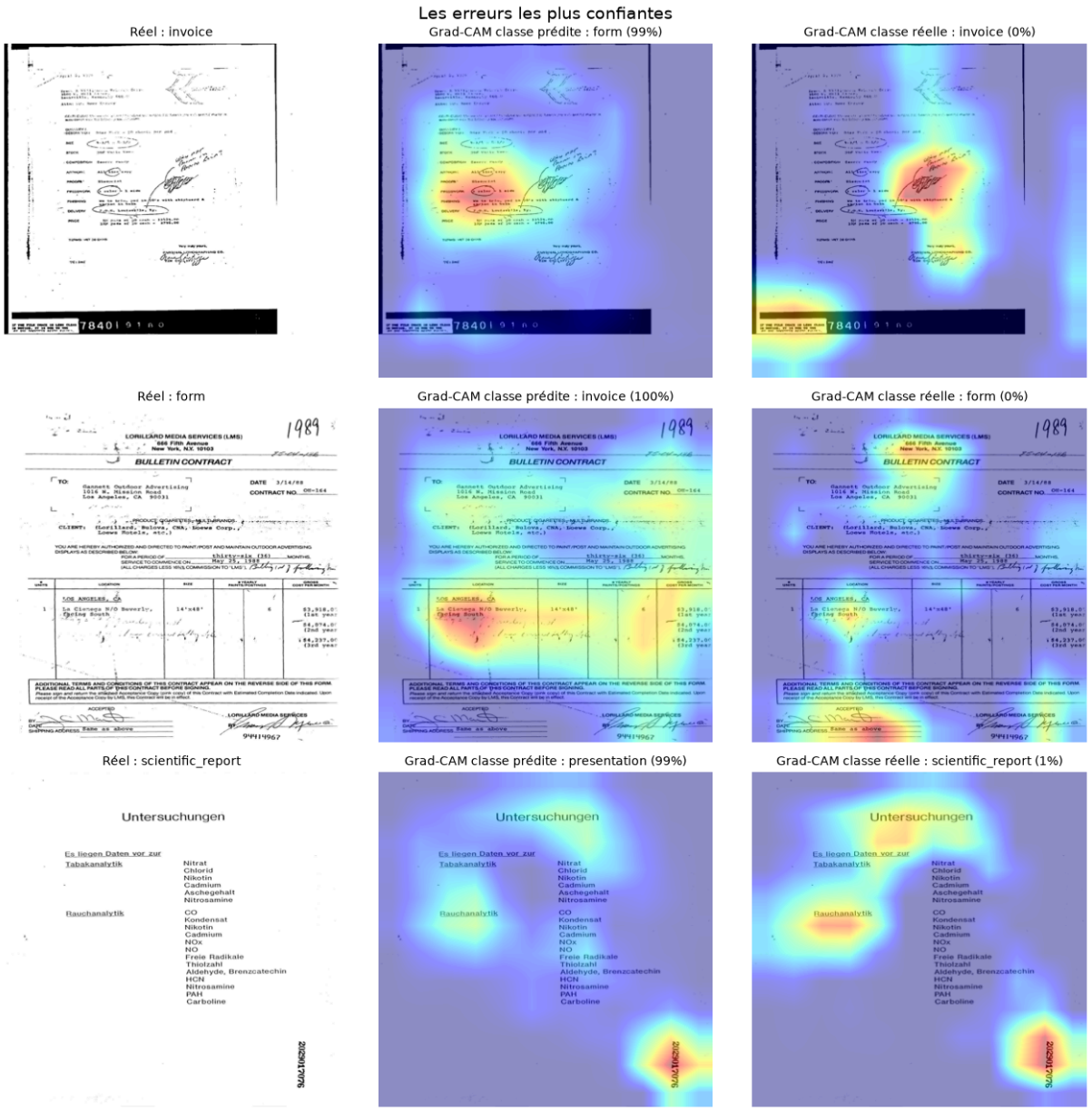


Source : 5_explicabilite/5_explicabilite_cnn.ipynb

Chaque filtre produit une copie du document. Les copies diffèrent parce que certaines font ressortir les traits horizontaux, d'autres les traits verticaux, d'autres encore l'encre sur fond clair ou l'inverse. À ce stade, le réseau ne voit que du contraste et des directions de traits, comme la première couche du cours ne voyait que des bords.

Dans les premières couches, les marges noires du scan ressortent fortement, parce que leur contraste est élevé.

Nous prenons trois erreurs où le CNN s'est trompé en étant presque sûr de lui. Pour chacune, nous affichons deux cartes : celle de la classe prédite, à tort, et celle de la vraie classe.



Source : 5_explicabilite/5_explicabilite_cnn.ipynb

Dans le cas de la facture (invoice) classée formulaire (form) à 99%, nous observons qu'il y a une colonne de libellés suivis de valeurs (size, stock, delivery, price). La zone rouge se pose exactement sur cette colonne, qui ressemble visuellement à un formulaire.

Dans le cas du formulaire (form) classé facture (invoice) à 100 %, il s'agit d'un contrat qui contient un tableau de montants. La zone rouge se pose sur ce tableau et le modèle applique 'tableau de montants = facture'.

Dans le cas du rapport scientifique (scientific_report) classé présentation (presentation) à 99 %, la page est presque vide, elle ne contient qu'un titre et deux listes. La zone rouge se pose en partie sur le numéro d'identification imprimé dans la marge.

5.2 SHAP

Grad-CAM s'appuie sur les gradients. SHAP procède autrement, il masque tour à tour des régions de l'image et mesure l'effet sur le score de chaque classe, selon les valeurs de Shapley. Une région rouge pousse vers la classe, une région bleue s'y oppose. Si deux méthodes aussi différentes désignent la même zone, l'explication est fiable.

Nous l'appliquons à deux documents : une facture bien classée et un contrat classé à tort en facture. Pour chacun, SHAP affiche les deux classes les plus probables, invoice et form.



Source : 5_explicabilite/5_explicabilite_cnn.ipynb

Dans le cas de la facture bien classée, les lignes du tableau poussent nettement vers invoice. Dans la colonne form, les contributions sont plus faibles et de signes opposés selon les zones. Même pour une facture reconnue à 100 %, la deuxième classe envisagée reste form.

Dans le cas du contrat classé invoice. Ce sont encore les lignes du tableau de montants qui poussent vers invoice. Ce qui plaide pour form est faible et dispersé sur la page, et ne suffit pas à inverser la décision.

Les deux méthodes s'accordent : c'est le tableau qui fait la décision, dans le bon cas comme dans le mauvais.

5.3 Constat

Le CNN lit la mise en page, pas le texte. Il reconnaît l'entête d'un email, des champs de formulaire, un tableau, une liste à puces ou un graphique.

Ses confusions viennent d'une vraie ambiguïté visuelle. Une facture et un formulaire peuvent avoir la même mise en page. Entraîner le CNN plus longtemps n'y changerait rien.

Nous avons repéré un biais, sur une page presque vide, le numéro imprimé dans la marge du scan.

Le CNN est trop sûr de lui. Il annonce 99 à 100 % de confiance même quand il se trompe. Sa confiance ne suffit donc pas à repérer ses erreurs.

6. Conclusion

Notre question de départ était de savoir, du texte ou de la mise en page, quelle source permet le mieux de reconnaître un document ?

L'explicabilité que nous avons réalisée montre que la mise en page permet d'atteindre un résultat car le CNN reconnaît un entête, un tableau ou une liste.

Mais le texte est la source la plus forte. Sur le premier extrait du dataset 2, la meilleure approche sur les pixels est à 50,6 % alors que le texte lu par OCR atteint 86,9 %.

Le modèle qui doit être retenu est celui qui combine nos 3 meilleurs modèles.

L'objectif de notre projet est atteint, nous avons un modèle qui obtient de bons résultats.

Nous tenons à remercier Nicolas, notre mentor tout au long du projet, qui a pu nous aiguiller et nous conseiller lors de nos doutes et décisions majeurs d'orientation de ce projet.